



Agentic Platform

PLATAFORMA AGÉNTICA MULTI-TENANT

MANUAL 10 · MANUAL DE USUARIO

Manual de desarrolladores: portal, API, SDKs, tutoriales y webhooks

Dirigido a: **Desarrolladores e integradores técnicos (con rol Tenant Admin para acuñar tokens y configurar webhooks)**

Este manual explica cómo integrar sistemas externos con la plataforma agéntica multi-tenant a través del **Portal de desarrollador** y del **visor de documentación** del panel de administración.

Recorrerás la página de inicio del portal, la referencia de la API pública v1 (OpenAPI 3.1 y Swagger UI), los SDKs oficiales de Python y TypeScript, los tutoriales paso a paso (acuñar un token, llamar a la API y configurar un webhook) y la guía de webhooks entrantes con firma HMAC. También se cubre el visor de documentación canónica del producto dentro del área /admin.

El Portal de desarrollador es público (no requiere sesión) y no hace llamadas a la API: es una capa fina que enlaza el contrato OpenAPI, los READMEs de los SDKs y la documentación canónica bajo /docs.

Generado · 2026-06-18

Idioma · Español

Versión · Producción

Confidencial · Comité de Dirección

Contenido

- 01** Visor de documentación (panel de administración)
- 02** Portal de desarrollador: inicio
- 03** API Reference: contrato OpenAPI y autenticación
- 04** SDKs oficiales: Python y TypeScript
- 05** Tutoriales: token, llamada a la API y webhook
- 06** Webhooks entrantes: orígenes, firma HMAC y checks

1 Visor de documentación (panel de administración)

Esta pantalla, dentro del área autenticada `/admin`, es el **visor de documentación** que permite explorar la documentación de cualquier proyecto del tenant en un solo lugar. La cabecera muestra el título **Documentación** con una breve descripción.

La columna izquierda tiene dos pestañas: **Explorar** y **Marcadores**. En **Explorar** encontrarás, de arriba a abajo: un buscador instantáneo de texto sobre los documentos, un panel de filtros por **categoría** y por **tipo** (que se activa al seleccionar un proyecto), y un árbol navegable de carpetas y archivos. Al seleccionar un documento, su ruta queda fijada en la URL (`?project=&path=`), de modo que el enlace es compartible y sobrevive a recargas.

Puedes marcar cualquier documento con la estrella desde el árbol, desde un resultado de búsqueda o desde la cabecera del visor; los marcados aparecen en la pestaña **Marcadores** (con un contador) y se guardan localmente en el navegador por tenant. El panel principal de la derecha renderiza el documento seleccionado en Markdown con su tabla de contenidos.

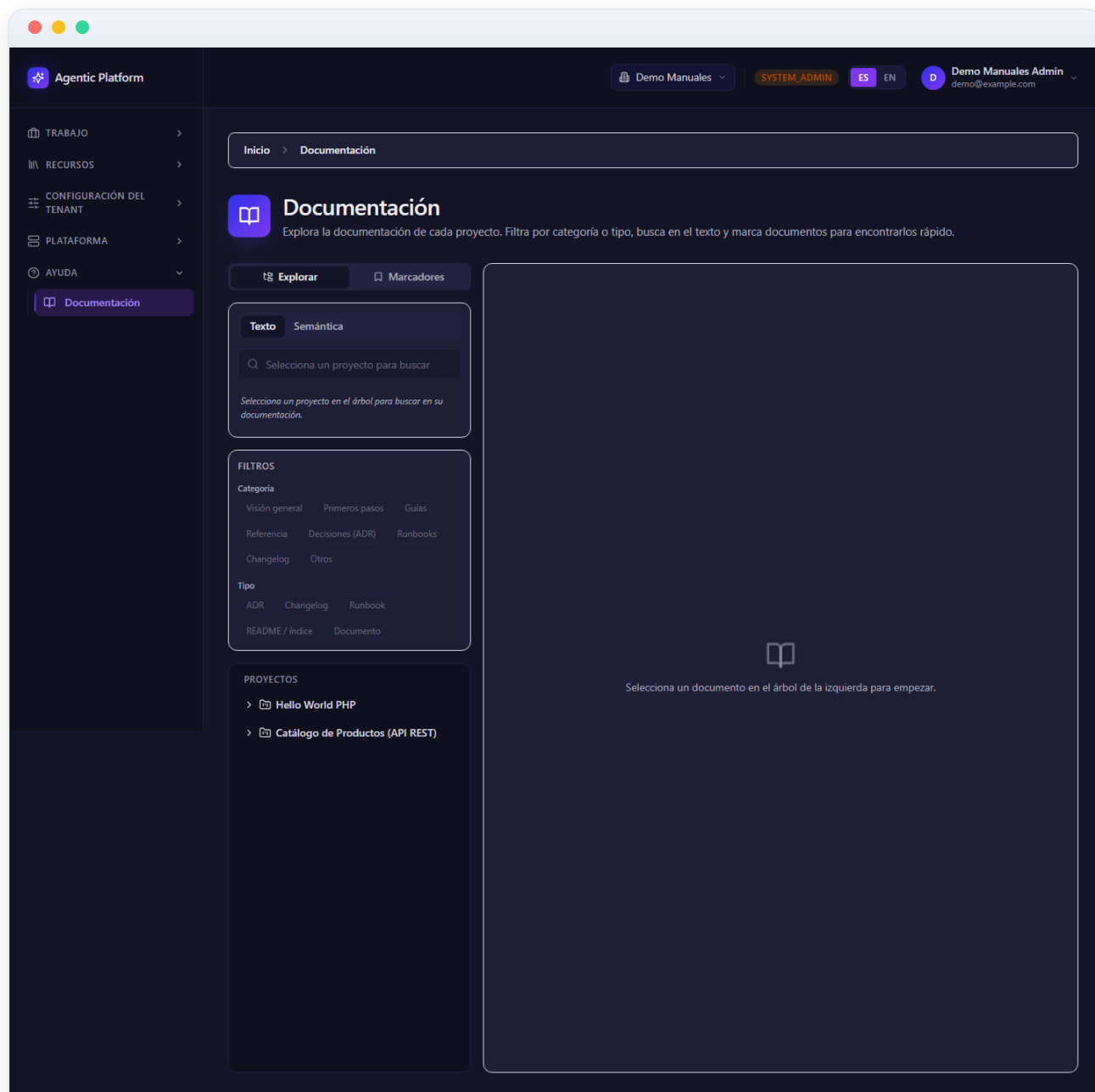


Figura 10.1 — Visor de documentación (panel de administración)

2 Portal de desarrollador: inicio

Esta es la página de entrada del **Portal de desarrollador**, una zona pública (fuera de `/admin`) que reúne todo lo necesario para integrar con la API pública v1. La cabecera incluye la marca **Portal de desarrollador** y una barra de navegación con: **Inicio**, **API Reference**, **SDKs**, **Tutoriales** y **Webhooks**.

El cuerpo presenta cuatro tarjetas enlazadas que resumen y dan acceso a cada sección: **API Reference** (contrato OpenAPI 3.1 y Swagger UI, autenticación por `X-API-Token`, scopes, rate limit y paginación), **SDKs oficiales** (clientes tipados de Python y TypeScript generados desde el OpenAPI), **Tutoriales** (tres pasos guiados) y **Webhooks entrantes** (orígenes soportados, firma HMAC-SHA256 y orden de checks).

Debajo, la sección **Documentación canónica** recuerda que la documentación completa del producto vive bajo `/docs` con la estructura de 7 carpetas y enlaza las referencias más útiles para integrar: la referencia API pública (`docs/04-reference/public-api.md`), la guía de integración (`docs/03-guides/api-publica-y-webhooks.md`) y los runbooks operativos (`docs/06-runbooks/`).

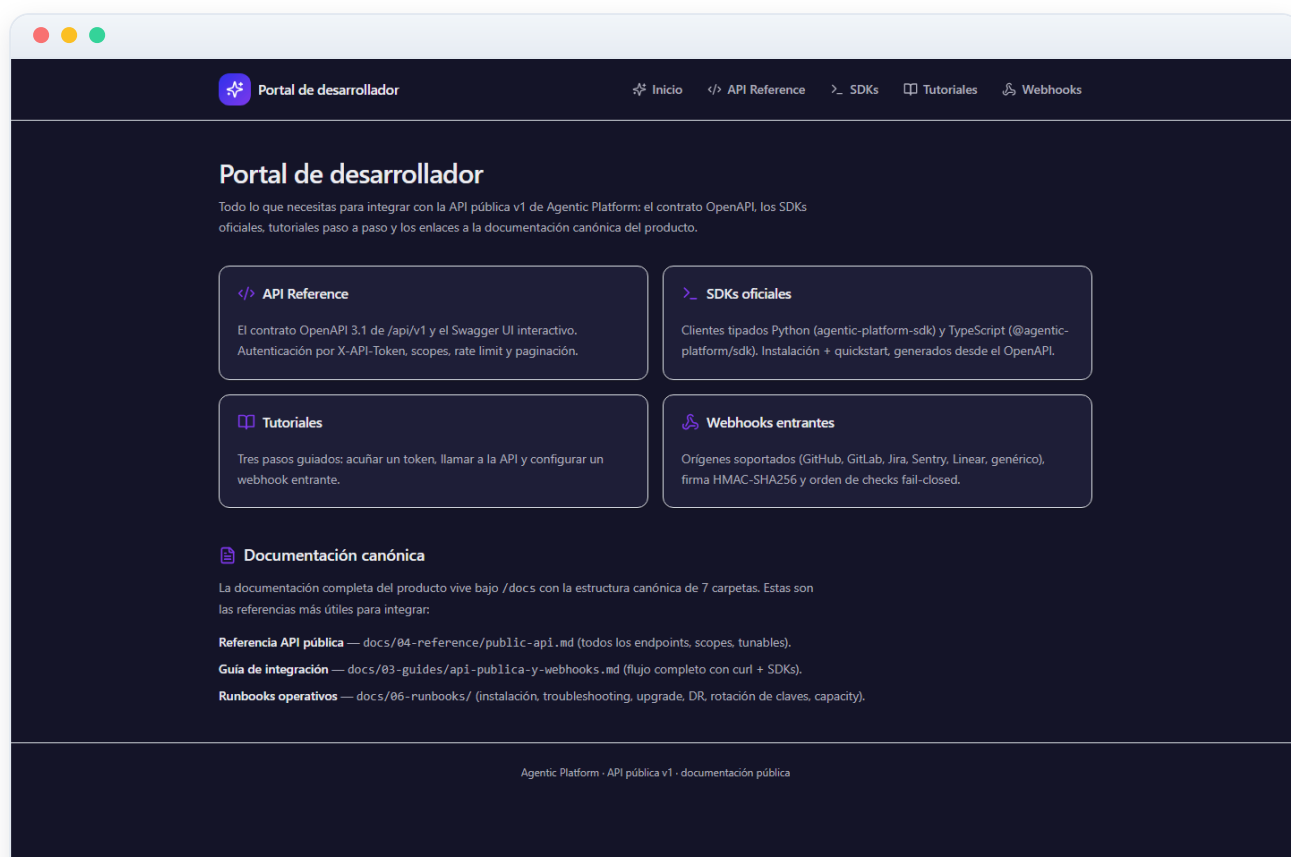


Figura 10.2 — Portal de desarrollador: inicio

3 API Reference: contrato OpenAPI y autenticación

Esta página documenta la **API pública v1** (</api/v1>), una fachada fina y versionada sobre el dominio. La primera tarjeta, **Contrato OpenAPI 3.1 + Swagger UI**, enlaza los dos recursos públicos que conviene leer antes de acuñar el primer token: </api/v1/openapi.json> (el contrato crudo, consumido por los SDKs) y </api/v1/docs> (el Swagger UI interactivo para explorar y probar). Debes sustituir la ruta por la URL pública de tu instalación.

La tarjeta **Autenticación, scope y aislamiento** explica las reglas clave: el token viaja siempre en la cabecera `X-API-Token` (nunca como query param); los `GET` requieren scope `read` y los `POST` requieren `write` (un `write` no concede `read` implícito); el aislamiento entre tenants lo garantiza PostgreSQL RLS (un id ajeno devuelve un `404` limpio); hay un rate limit por token (100 req/min por defecto) con cabeceras `X-RateLimit-*`; y todas las listas son paginadas con `limit` / `offset`. Incluye un ejemplo `curl`.

Más abajo, la tabla **Endpoints (scope mínimo)** lista las rutas disponibles (projects, plans, tasks, conversations y knowledge bases) con su método y scope requerido, y la tabla **Códigos de estado** describe el significado de 200/201, 400, 401, 403, 404 y 429.

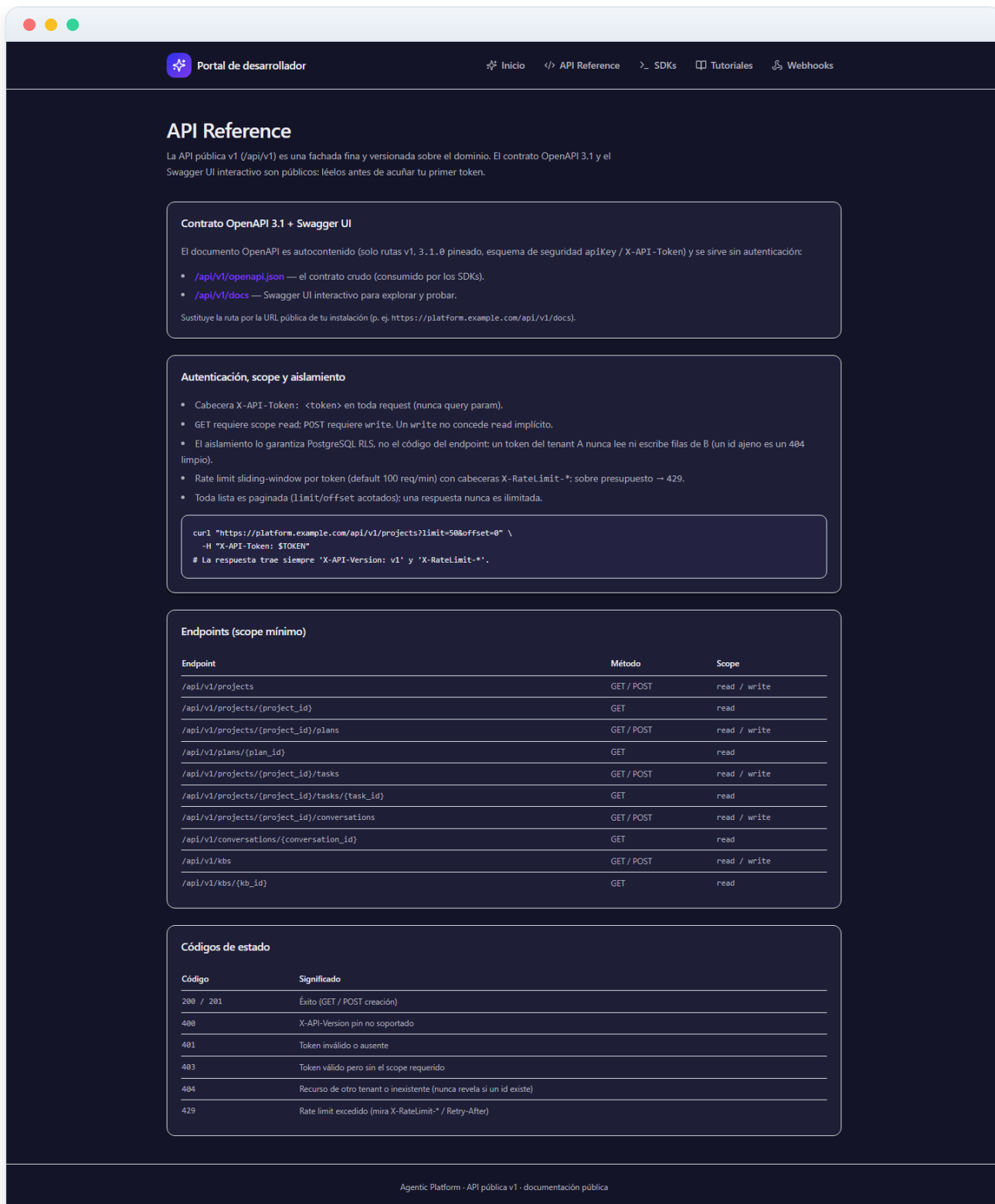


Figura 10.3 — API Reference: contrato OpenAPI y autenticación

4 SDKs oficiales: Python y TypeScript

Esta página describe los dos **SDKs oficiales** tipados sobre la API pública v1. Ambos se generan desde el contrato OpenAPI 3.1 en proceso (sin servidor vivo), de modo que siempre casan con el servidor: modelos generados más un cliente fino que fija el `X-API-Token` una vez y eleva errores tipados.

La tarjeta **SDK Python — `agentic-platform-sdk`** indica sus dependencias de runtime (`httpx` + `pydantic` v2), muestra el bloque de **Instalación** (`pip install` desde el monorepo o desde el registro interno) y un **Quickstart** con código de ejemplo para listar, obtener y crear proyectos y para consultar plans, tasks (project-scoped) y knowledge bases (tenant-scoped). Una respuesta non-2xx eleva `ApiError` con `status_code` y `body` para ramificar por 401, 403, 404 o 429.

La tarjeta **SDK TypeScript — `@agentic-platform/sdk`** destaca que tiene cero dependencias de runtime (usa `fetch` nativo, Node 18+ o navegador) y muestra su instalación y quickstart equivalentes. La última tarjeta, **Regeneración (reproducibilidad)**, explica cómo regenerar los modelos tras cambiar el contrato ejecutando los scripts `generate` de cada SDK.

Portal de desarrollador

Inicio

</> API Reference

> SDKs

Tutoriales

Webhooks

SDKs oficiales

Dos clientes tipados sobre la API pública v1. Ambos se generan DESDE el OpenAPI 3.1 en proceso (sin servidor vivo), así que siempre casan con el servidor: modelos generados + un cliente fino escrito a mano que fija X-API-Token una vez y eleva errores tipados.

SDK Python — agentic-platform-sdk

Dependencias de runtime: httpx + pydantic v2. El token viaja en la cabecera X-API-Token en cada request.

Instalación

```
pip install -e packages/sdk-python      # desde el monorepo
# o, una vez publicado en el registro interno:
pip install agentic-platform-sdk
```

Quickstart

```
from agentic_platform_sdk import ApiClient, V1ProjectCreateRequest

# URL base de la plataforma + un X-API-Token por tenant.
with ApiClient("https://platform.example.com", "tkn...") as api:
    # listar (paginado)
    for project in api.list_projects(limit=50, offset=0):
        print(project.id, project.name, project.status)

    # obtener uno
    project = api.get_project("11111111-1111-1111-1111-111111111111")

    # crear (requiere un token con scope write)
    created = api.create_project(V1ProjectCreateRequest(name="Mi proyecto"))

    # plans / tasks / conversations son project-scoped
    plans = api.list_plans(created.id)
    tasks = api.list_tasks(created.id)

    # las knowledge bases son tenant-scoped
    kbs = api.list_kbs()
```

Una respuesta non-2xx eleva agentic_platform_sdk.ApiError con status_code + body: ramifica por 401 (token malo), 403 (scope), 404 (cross-tenant) o 429 (rate limit).

SDK TypeScript — @agentic-platform/sdk

Cero dependencias de runtime: usa el fetch de la plataforma (Node 18+ / navegador); se puede inyectar un fetch propio para tests.

Instalación

```
npm install # dentro de packages/sdk-typescript (workspace)
```

Quickstart

```
import { ApiClient, type V1ProjectCreateRequest } from "@agentic-platform/sdk";

// URL base de la plataforma + un X-API-Token por tenant.
const api = new ApiClient({
  baseUrl: "https://platform.example.com",
  apiToken: "tkn...",
});

// listar (paginado)
const projects = await api.listProjects({ limit: 50, offset: 0 });

// crear (requiere un token con scope write)
const created = await api.createProject({ name: "Mi proyecto" } satisfies V1ProjectCreateRequest);

// plans / tasks / conversations son project-scoped
const plans = await api.listPlans(created.id);

// las knowledge bases son tenant-scoped
const kbs = await api.listKbs();
```

Una respuesta non-2xx lanza ApiError con statusCode + body (mismas ramas 401 / 403 / 404 / 429).

Regeneración (reproducibilidad)

Tras cualquier cambio del contrato público, regenera los modelos desde el OpenAPI v1 construido en proceso:

```
python packages/sdk-python/scripts/generate.py
node packages/sdk-typescript/scripts/generate.mjs
```

El código generado se excluye de los linters; el test de cada SDK (paridad modelo ≠ schema, cabecera X-API-Token, errores tipados) sí corre en CI.

Agentic Platform · API pública v1 · documentación pública

Figura 10.4 — SDKs oficiales: Python y TypeScript

5 Tutoriales: token, llamada a la API y webhook

Esta página recoge **tres tutoriales paso a paso** para pasar de cero a integrado. Requieren el stack en marcha y el rol **Tenant Admin** (acuñar tokens y gestionar webhooks lo exigen). Recuerda sustituir `platform.example.com` por la URL pública de tu instalación.

El paso **1 · Acuñar un X-API-Token** muestra cómo crear la credencial por tenant con `POST /auth/api-tokens`, indicando que el token claro se devuelve una sola vez (hay que guardarlo) y explicando los campos: `scopes` (`read` solo permite GET, añade `write` para crear), y los opcionales `rate_limit`, `expires_at` e `ip_allowlist`; la revocación se hace con `DELETE /auth/api-tokens/{token_id}`, efectiva de inmediato.

El paso **2 · Llamar a la API v1** muestra ejemplos `curl` para listar y crear proyectos con el token en la cabecera `X-API-Token`, recordando las reglas de scope y los códigos 404/429. El paso **3 · Configurar un webhook entrante** muestra cómo crear la configuración con `POST /projects/{project_id}/incoming-webhooks`, que devuelve el `incoming_path` y el `signing_secret` en claro una sola vez. La página enlaza a las secciones de SDKs y Webhooks para profundizar.



Figura 10.5 — Tutoriales: token, llamada a la API y webhook

6 Webhooks entrantes: orígenes, firma HMAC y checks

Esta página documenta los **webhooks entrantes**: un tool externo hace POST de un evento firmado con HMAC y el sistema lo verifica y lo mapea a una acción. El endpoint es público, por lo que **la propia firma HMAC actúa como autenticación**.

La tarjeta **Orígenes soportados (catálogo cerrado)** incluye una tabla con cada **origin** (github, gitlab, jira, sentry, linear y generic), la cabecera de firma que espera y los eventos típicos que generan acciones (crear tarea, comentar o escalar). Todas las firmas son HMAC-SHA256 sobre el body crudo, comparadas en tiempo constante; el endpoint público es **POST** `/webhooks/incoming/{origin}/{config_id}`.

La tarjeta **Orden de checks (fail-closed)** enumera los seis pasos que sigue el sistema: límite de tamaño del body (413), resolver la config (404), rate limit por config (429), verificar la firma HMAC (401, sin acción si falla), mapear y actuar, y persistir de forma idempotente. La última tarjeta, **Verificar la firma manualmente (genérico)**, muestra cómo calcular la firma con **openssl** y enviarla con **curl**, e indica que la rotación del secreto se hace con **POST** `.../incoming-webhooks/{config_id}/rotate-secret`, que invalida el secreto anterior de inmediato.

Portal de desarrollador

[Inicio](#) [API Reference](#) [SDKs](#) [Tutoriales](#) [Webhooks](#)

Webhooks entrantes

La dirección inversa del firmado saliente: un tool externo hace POST de un evento firmado con HMAC y el sistema lo verifica y lo mapea a una acción. El endpoint es público — la HMAC ES la autenticación.

Orígenes soportados (catálogo cerrado)

origin	Cabecera de firma	Eventos típicos
github	X-Hub-Signature-256: sha256=<hex>	push, PR review → crear/actualizar tarea
gitlab	X-Hub-Signature-256: sha256=<hex>	merge request → crear tarea
jira	X-Signature-256: <hex> (bare)	issue creado → crear tarea
sentry	X-Signature-256: <hex> (bare)	error → crear bug task / escalar
linear	X-Signature-256: <hex> (bare)	issue → crear tarea
generic	X-Signature-256: <hex> (bare)	integración a medida / proxy normalizador

Todas las firmas son HMAC-SHA256 sobre el body crudo, comparadas en tiempo constante. El endpoint público es POST `/webhooks/incoming/{origin}/{config_id}`.

Orden de checks (fail-closed)

- 1 • Body cap (413)** — Antes de leer el body (guarda anti-DDoS; default 1 MiB).
- 2 • Resolver config (404)** — `config_id` → `file`. Inexistente / `soft-deleted` / `deshabilitada` / con `origin` que no casa → 404 (nunca revela si un id existe).
- 3 • Rate limit por config (429)** — Sliding-window keyed por `config_id` (default 120/min); una config nunca throttlea a otra.
- 4 • Verificar HMAC (401, SIN acción)** — Recomputa el MAC con el secreto por proyecto y compara en tiempo constante. Firma mala/ausente/manipulada → 401, nada persistido.
- 5 • Mapear + actuar** — Normaliza el payload (plantilla del origen) y resuelve `action_mappings` → acción (crear tarea / comentar / escalar), en la misma transacción que registra el evento.
- 6 • Persistir (idempotente)** — UNIQUE parcial (`config_id`, `delivery_id`); una redelivery colisiona, así que ni el evento ni su acción se reaplican.

Verificar la firma manualmente (genérico)

```
BODY='{"hello":"world"}'
SIG=$(printf '%s' "$BODY" | openssl dgst -sha256 -hmac "$SECRET" -hex | sed 's/^.* //' )
curl -X POST https://platform.example.com/webhooks/incoming/generic/$CONFIG_ID \
  -H "Content-Type: application/json" \
  -H "X-Signature-256: $SIG" \
  -d "$BODY"
# → 202 { "status": "accepted", "event_id": "...", "action": "create_task", "task_id": "..." }
```

Rotación: si sospechas que el secreto se filtró, POST `.../incoming-webhooks/{config_id}/rotate-secret` devuelve un nuevo claro una vez; el anterior deja de verificar de inmediato.

Agentic Platform · API pública v1 · documentación pública

Figura 10.6 — Webhooks entrantes: orígenes, firma HMAC y checks